ClientLens — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and the one boundary

A Supervisor-led team over two MCP services. Reuse what you've built:

- `servers/orders_server.py` + `servers/tickets_server.py` — tiny `MCPServer`s (`get_orders`, `get_tickets` over a mocked dict).
- `src/mcp_client.py` — `run_specialist(client, command, args, system, task)`: open `stdio_client` / `ClientSession`, `initialize()`, `list_tools()`, run a ReAct loop, return `(findings, unavailable)`.
- `src/agents/orders_agent.py`, `tickets_agent.py` — thin wrappers calling `run_specialist` with their server.
- `src/agents/supervisor.py` — `decide_next` + `supervisor` (`Command`).
- `src/agents/writer_agent.py` — `run_writer` (plain LLM, synthesize).
- `src/graph.py` — `START -> supervisor`; async specialist nodes; `.ainvoke()`.

The boundary that matters: **the Supervisor only routes** (`Command(goto=...)`); it never touches a `ClientSession`. Each specialist owns its own connection — so one service's failure can't touch another's.

Hint 2 — the shared specialist loop, and degradation

```
run_specialist(client, command, args, system, task):
    try:
        async with stdio_client(params) as (r,w):
            async with ClientSession(r,w) as s:
                await asyncio.wait_for(s.initialize(), timeout=...)
                tools = list_tools() -> openai schema
                loop: create(tools=...) -> run tool calls -> repeat
                return (answer, False)
    except Exception:
        return ("", True)         # degradation lives HERE, inside the node
```

The connection opens and closes inside this one async call, so a specialist runs cleanly as a graph node (its teardown stays in one task). And the `try/except` around the whole thing is the degradation: a dead service returns `("", True)` instead of crashing the graph.

Hint 3 — the Supervisor and the async graph

```
decide_next(state):
    if needs_orders(task) and orders == "":   return "orders"
    if needs_tickets(task) and tickets=="" and not tickets_unavailable:
        return "tickets"
    return "writer"
supervisor(state):
    Command(goto=decide_next(state), update={routing_log+[target]})
orders_node/tickets_node: async; await run_*_agent(...)
    -> Command(goto="supervisor", update=...)
run the graph with: await app.ainvoke(start_state(task))
```

The one thing people get wrong: specialist nodes must be `async def` and the graph run with `.ainvoke()`. Don't wrap the async specialist in `asyncio.run()` inside a sync node — it fights LangGraph's loop and hangs. Note `decide_next` routes to `writer` the moment `tickets_unavailable` is true, so the team never loops on a dead service.

Hint 4 — the Writer, and proving degradation

The Writer is a plain LLM call that weaves both findings into **one** 360 summary and reads `tickets_unavailable` to name a missing source:

```
run_writer(client, task, orders, tickets, tickets_unavailable):
    tickets_section = tickets
    if tickets_unavailable: tickets_section = "(tickets unavailable)"
    system = "Synthesize into ONE 360; weave orders+tickets; if a source is
              unavailable, say so; never pretend it was checked."
    return create([system, user(task, orders, tickets_section)]).content
```

Prove degradation: give `build_graph` an optional `tickets_args` and point it at a script that doesn't exist. The run finishes from orders alone, `tickets_unavailable` is `True`, and the summary names the gap. Catching **inside** the specialist node (not the Supervisor or the graph) is what keeps one dead service from touching the rest of the team.

Still stuck? The whole project, every file, is in the complete solution.